

ADR-004 — Estrategia de despliegue de los servicios del sistema

ID	Título	Estado	Fecha
ADR-004	Estrategia de despliegue de los servicios del sistema	Aceptada	27/05/2026

1. Contexto

La división en dos servicios definida en ADR-001 obliga a decidir cómo y dónde se despliega cada uno. Los dos servicios tienen perfiles operativos distintos:

- `my-app` es Next.js 16 con App Router, Server Actions y Prisma sobre Postgres. Su carga es interactiva (sesiones de productor durante horario laboral), con picos cortos y mucho tiempo ocioso entre acciones.
- `chatbot` es FastAPI sobre Python 3.13 con SQLite local, un webhook público que Meta llama cada vez que un cliente escribe, y estado conversacional que vive en BD local entre turnos.

Motivadores que obligan a tomar una decisión arquitectónica de despliegue:

- Motivador 1 — Disponibilidad 24/7 (RNF Entrega 1):** el bot debe poder ser invocado por Meta en cualquier momento; la URL pública debe ser estable y certificada (HTTPS). El panel debe estar disponible al menos en horario extendido para productores.
- Motivador 2 — Restricciones técnicas opuestas entre los dos servicios:** el panel encaja muy bien en plataformas serverless por su perfil bursty y por la integración nativa Next.js + Vercel + Supabase. El bot, en cambio, **no** encaja en serverless: tiene BD local (SQLite en `/data/chatbot.db`), webhook con sesión HTTPS, y un estado conversacional que se perdería entre invocaciones efímeras.
- Motivador 3 — Capacidad del equipo (sin DevOps dedicado):** ningún integrante administra clusters de Kubernetes ni VMs en producción. La operación tiene que reducirse al mínimo: configurar, conectar el repo y olvidar.
- Motivador 4 — Costo:** proyecto académico/PoC, sin presupuesto para infraestructura on-premise ni planes empresariales.

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
Todo en una VPS con Docker Compose (un VPS aloja panel + bot + Postgres)	Un solo lugar a operar, costos predecibles, control total.	Contradice los motivadores 1 y 3: cualquier reinicio de la VM tira los dos servicios a la vez; requiere que alguien del equipo opere la VM (backups, certificados TLS, monitoring). Pierde la integración nativa <code>Next.js↔Vercel↔Supabase</code> .
Todo en Kubernetes (clúster propio o gestionado)	Despliegues canarios, escalado horizontal, networking sofisticado.	Sobreingeniería absoluta para dos servicios y un equipo de 6 personas. Curva de operación demasiado pronunciada (motivador 3); el costo de un clúster gestionado es alto (motivador 4).

Vercel para el panel + PaaS de contenedores para el bot (Fly.io / Railway / Render) (elegida)	Cada servicio en la plataforma que mejor le sirve; ambas plataformas se gestionan por dashboard y conectan al repo de Git. Cero servidores que administrar.	— (esta fue la elegida)
---	---	-------------------------

3. Decisión tomada

Se decide: desplegar `my-app` en **Vercel** (con la base de datos PostgreSQL administrada por Supabase, ver ADR-002) y `chatbot` como contenedor en un **PaaS de contenedores** (Fly.io o Railway), expuesto vía URL pública con HTTPS provista por la plataforma. Cada servicio tiene su propio pipeline conectado al monorepo (filtrado por path).

Fundamentación:

- Resuelve el motivador 1 (disponibilidad y URL pública estable):** Vercel da TLS y URL estable para el panel automáticamente; el PaaS de contenedores da una URL `https://...` estable que se registra como webhook en Meta. Ambos servicios escalan/recuperan sin intervención manual.
- Resuelve el motivador 2 (restricciones técnicas opuestas):** Vercel está hecho para Next.js (server components, edge caché, preview deploys por PR) y se integra nativamente con Supabase. Un PaaS de contenedores corre el bot como proceso persistente con volumen para `/data/chatbot.db`, preservando estado entre requests; ningún integrante toca un Dockerfile complejo.
- Resuelve los motivadores 3 y 4 (sin DevOps, costo bajo):** ambas plataformas tienen tier gratuito o muy barato suficiente para un PoC; la operación se reduce a configurar variables de entorno y revisar logs en el dashboard. Cero servidores propios, cero certificados manuales.

4. Consecuencias

Consecuencias positivas	Trade-offs / costos
Cada servicio en la plataforma que mejor encaja con su runtime: deploys rápidos en ambos lados.	Dos paneles de operación distintos (Vercel + PaaS de contenedores). Las métricas, logs y alertas no están unificadas.
Preview deploys del panel en cada PR sin trabajo adicional (Vercel nativo).	Los secretos (URLs de Supabase, X-API-Key de ADR-003, credenciales de Meta de ADR-005) se gestionan en dos lugares , con riesgo de divergencia entre entornos.
Vendor de hosting está atado al stack, pero ambos son sustituibles: el bot vive en un contenedor estándar y el panel es Next.js puro.	El bot mantiene estado local en SQLite : si el contenedor se mueve de nodo sin volumen persistente, se pierde el estado conversacional en curso. Hay que asegurar configuración de volumen en la plataforma elegida.
Disponibilidad por servicio: caída del bot no afecta al panel y viceversa, alineado con ADR-001 y ADR-003.	Latencia de red entre el bot y el panel: las llamadas REST cruzan internet entre dos PaaS distintos. Tolerable para el patrón request/response definido en ADR-003, pero hay que medirla.

Decisiones relacionadas:

- ADR-001 (separación de servicios): este ADR es la materialización operativa de aquella decisión.
- ADR-002 (PostgreSQL en Supabase): la elección del proveedor de BD encaja con Vercel.
- ADR-003 (REST con API Key): el bot necesita conocer la URL pública del panel; esa URL la provee Vercel.